

CAMINO DE UNIFICACIÓN

Qué cambiar en Maily te cuida para unificarlo

Lista concreta y accionable de los cambios necesarios para que el backend actual de Maily te cuida (app del paciente) deje de ser un sistema aislado y se integre en la Maily Platform unificada: multi-tenant, identidad global del paciente, especialidades como plugins, autenticación y cobro unificados.

SECCIÓN 1

Objetivo y alcance

El backend actual es **de buena calidad (7/10) y está ~80% completo**, pero fue construido como una app de paciente *aislada* (B2C, sin multi-tenant, especialidades fijas, auth con Clerk). Este documento detalla lo que hay que transformar para que sea parte de **una sola plataforma** donde clínica (Maily Soft) y paciente (Maily te cuida) comparten cimientos.

Recordatorio honesto

Unificar es un **refactor mayor** (semanas, no días). La alternativa —mantener dos backends separados que se comunican por API— es más rápida pero te deja dos sistemas que mantener. Este documento asume que **eliges unificar**. Si dudas, lee primero la Sección 5 (modelo de negocio) y la 6.

Leyenda de esfuerzo por cambio: **BAJO** (días) · **MEDIO** (1-2 semanas) · **ALTO** (2+ semanas)

SECCIÓN 2

Esfuerzo total estimado

Bloque de cambios	Esfuerzo	Riesgo
C1 · Modelo base abstracto + multi-tenant (tenant_id en todo)	ALTO	Alto (toca todos los modelos + migraciones)
C2 · Identidad global del paciente (MPI)	ALTO	Alto (cambia el eje del sistema)
C3 · Especialidades a sistema de plugins	MEDIO	Medio
C4 · Auth unificada (Clerk → JWT o abstracción)	MEDIO	Medio
C5 · Modelo de negocio unificado (B2B + B2C)	MEDIO	Medio
C6 · Capa de servicios/selectors (alinear a estándares)	MEDIO	Bajo (gradual)
C7 · Resolver solapamiento de apps con Maily Soft	ALTO	Alto (definir fuente de verdad)
C8 · Correcciones de seguridad (bloqueantes)	BAJO	Crítico si no se hace
C9 · Calidad y herramientas (mypy, N+1, factories)	BAJO	Bajo

Estimación realista

Con un equipo de 2 devs senior: **6 a 10 semanas** de refactor para tener la base unificada estable. La seguridad (C8) se hace en **la primera semana** sí o sí. C1, C2 y C7 son el grueso del trabajo y deben ir primero porque todo lo demás depende de ellos.

SECCIÓN 3

Mapa de brechas: hoy vs. objetivo

Dimensión	Hoy (Maily te cuida)	Objetivo (Maily Platform)
Multi-tenant	No existe	Toda entidad con <code>tenant_id</code> + RLS
Identidad del paciente	1:1 usuario↔paciente, una sola "clínica"	Global (MPI), se conecta a varias clínicas
Especialidades	Apps fijas hardcodedas	Plugins + JSON Schema por especialidad
Autenticación	Clerk (acoplado)	JWT propio unificado (o Clerk tras abstracción)
Cobro	B2C mensual (FREE/SILVER/GOLD/PLATINUM)	B2B anual (clínica) + B2C opcional (paciente premium)
Arquitectura de código	Lógica en views/serializers	Capa de servicios/selectors
Modelo base	Cada modelo repite id/timestamps	<code>TenantAwareModel</code> abstracto común
Tipado / mypy	Parcial, sin mypy	Type hints + mypy en CI

CAMBIO 1 · ALTO

C1. Modelo base abstracto + multi-tenant

Es el cimientto de todo. Hoy no hay multi-tenancy: ninguna tabla sabe a qué clínica pertenece. Hay que introducir un modelo base que TODA entidad de negocio herede.

QUÉ HACER

1. Crear `core/models.py` con un modelo abstracto `TenantAwareModel`.
2. Migrar cada modelo de negocio para que herede de él (agrega `tenant_id`, timestamps, soft delete).
3. Agregar un `TenantManager` que filtre automáticamente por la clínica del usuario.
4. Activar Row Level Security en PostgreSQL como segunda barrera.
5. Middleware que fije el tenant del request a partir del token.

```
# core/models.py - NUEVO
class TenantAwareModel(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid4, editable=False)
    tenant_id = models.UUIDField(db_index=True) # ← LA clave nueva
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    deleted_at = models.DateTimeField(null=True, blank=True)

    objects = TenantManager() # filtra por tenant automático
    all_objects = models.Manager() # sin filtro (admin)

class Meta:
    abstract = True
```

El reto real

No es escribir el modelo base (eso es fácil) — es **migrar los datos existentes** asignándoles un `tenant_id` y reescribir cada query para respetar el tenant. Por eso es ALTO. Hazlo módulo por módulo, no todo de golpe.

CAMBIO 2 · ALTO

C2. Identidad global del paciente (Master Patient Index)

Hoy `PatientProfile` es un `OneToOne` con el usuario y asume una sola relación. En la plataforma, el paciente es **global** y se conecta a varias clínicas.

QUÉ HACER

- Separar la identidad global (`PatientAccount` , sin tenant) del vínculo por clínica (`ClinicPatientLink` , con tenant).
- Mover el perfil de salud "propio del paciente" (alergias, tipo de sangre, crónicos) a la cuenta global.
- Usar **CURP** como llave anti-duplicados.
- Los datos clínicos (vitals, medications, lab_results...) cuelgan del `ClinicPatientLink` , no del paciente directo.

HOY: User —1:1— PatientProfile —< vitals, medications, ...

OBJETIVO:

```
PatientAccount (global, sin tenant)      ← identidad + salud propia + gamificación
    | se conecta a (1:N)
    ▼
ClinicPatientLink (con tenant_id)         ← uno por clínica
    |
    ▼
vitals, medications, lab_results, ...     ← cuelgan del link (tenant-scoped)
```

Impacto fuerte

Esto cambia el "eje" del sistema. Casi todas las apps de datos clínicos (vitals, medications, prescriptions, lab_results, documents) deben repuntar de `patient` a `patient_link`. Es de los cambios más invasivos — planéalo con migración de datos y doble lectura.

CAMBIO 3 · MEDIO

C3. Especialidades a sistema de plugins

Hoy nutrición, fisio, etc. son apps Django fijas. En la plataforma deben ser **plugins activables por clínica**, con su hoja de datos definida por JSON Schema.

QUÉ HACER

- Crear un registro de especialidades (`Specialty`) y un contrato `SpecialtyPlugin` .
- Las hojas de cada especialidad se definen por **JSON Schema** y se guardan en un campo `specialty_data` (JSONB) del `encounter`, validado contra el schema.
- La especialidad se activa por tenant (y es un add-on cobrable).
- Mantener `apps/nutrition` como la **implementación de referencia** del plugin (no tirarla — adaptarla al contrato).

Lo bueno

No hay que reescribir la lógica de nutrición; hay que *envolverla* en el contrato de plugin. El núcleo clínico (`encounter` común) se comparte, y solo la "hoja" varía por especialidad. Ver el documento de arquitectura, Sección 8.

CAMBIO 4 · MEDIO

C4. Autenticación unificada (Clerk → JWT)

Hoy todo pasa por Clerk (middleware, webhooks, sync de usuarios). La plataforma usa **un solo sistema de auth** para clínica y paciente.

QUÉ HACER (2 OPCIONES)

Opción	Qué implica	Cuándo
A. Migrar a JWT propio (<code>simplejwt</code>)	Quitar Clerk, implementar login/registro/refresh propios. Más control, sin dependencia externa, alineado al diseño.	Recomendado si Maily Soft también usará JWT propio.
B. Abstraer auth detrás de una interfaz	Dejar Clerk para pacientes pero detrás de una capa <code>auth provider</code> , para poder cambiarlo sin tocar el resto. ~3-4 h de desacople inmediato.	Si quieres avanzar ya y decidir después.

Primero, sin importar la opción

Corregir **YA** la validación del JWT de Clerk: hoy no valida `iss` ni `azp` (acepta tokens de otras instancias). Es un hallazgo de seguridad ALTO (ver C8).

CAMBIO 5 · MEDIO

C5. Modelo de negocio unificado

Hay un choque: hoy el cobro es **B2C mensual** (el paciente paga FREE/SILVER/GOLD/PLATINUM); el diseño de la plataforma es **B2B anual** (la clínica paga + add-ons por especialidad).

QUÉ HACER: NO FUSIONAR, COEXISTIR (COBRO DUAL)

Flujo	Quién paga	Qué
B2B (Maily Soft)	La clínica	Suscripción anual + add-ons por especialidad/almacenamiento. Es el ingreso principal.
B2C (Maily te cuida)	El paciente (opcional)	Premium: IA avanzada, más almacenamiento de estudios, features de bienestar. Ingreso secundario.

- Renombrar/reubicar el modelo de planes del paciente para que sea claramente "premium del paciente", no el plan de la clínica.
- El `ai_engine` que asigna modelo por tier (SILVER→gpt-4o, PLATINUM→claude) se mantiene, pero como beneficio B2C.
- Unificar la infraestructura de cobro (Stripe) para que ambos flujos usen el mismo motor de suscripciones y webhooks.

Buena noticia

Tener dos fuentes de ingreso (clínica + paciente premium) es una **ventaja**, no un problema. Solo hay que separarlas conceptualmente en el modelo de datos.

CAMBIO 6 · MEDIO

C6. Capa de servicios y selectors

Hoy la lógica vive en views/serializers (consistente, pero fuera del estándar de la plataforma). Hay que migrar gradualmente a la arquitectura por capas de la skill `django-clean-architecture`.

QUÉ HACER

- Por cada app, extraer la lógica de negocio a `services.py` (escrituras) y `selectors.py` (lecturas).
- Las views quedan delgadas: validan entrada, llaman al servicio, devuelven respuesta.
- Usar como modelo a seguir `apps/notifications/service.py` y `apps/wellness/services.py` (ya están bien hechos).
- Es un cambio **gradual**: no bloquea, se hace app por app conforme se tocan.

CAMBIO 7 · ALTO

C7. Qué pasa con cada app (la decisión clave)

Varias apps **se solapan** con lo que hará el backend de la clínica (Maily Soft). Para cada una hay que decidir la **fuentes de verdad**: quién es dueño del dato.

App actual	Disposición	Detalle
accounts	 Fusionar	Se integra al sistema de usuarios + tenancy unificado.
PatientProfile	 Refactor	Se divide en <code>PatientAccount</code> (global) + <code>ClinicPatientLink</code> (tenant). Ver C2.
prescriptions	 Dueño: Maily Soft	La clínica EMITE recetas; el paciente las VE. El paciente deja de "crear", solo consume.
appointments	 Dueño: Maily Soft	La agenda vive en la clínica. El paciente agenda contra la agenda de la clínica.
specialists	 Fusionar	Se convierte en parte del directorio + plugins de especialidad. Ver C3.
lab_results / documents	 Dueño: Maily Soft	Datos clínicos tenant-scoped. El paciente los ve vía su link.
medications / vitals	 Compartida	Origen mixto: la clínica indica, el paciente registra adherencia/mediciones. Tenant-scoped.
payments	 Fusionar	Motor de cobro unificado B2B + B2C. Ver C5.
nutrition	 A plugin	Implementación de referencia del sistema de especialidades.
chat / telemedicine	 Compartida	Paciente↔doctor. Necesita contexto de tenant.
notifications / audit	 Infraestructura	Servicios transversales. Se reutilizan tal cual (audit alimenta NOM-024).
wellness / gamification / family_care / surveys / coupons / partners	 Se quedan (paciente)	Son features de engagement del paciente. Solo se les agrega tenant-awareness donde aplique.
analytics	 Dividir	Analítica de paciente (se queda) vs. analítica de clínica (a Maily Soft).

Regla para decidir el dueño

Si el dato lo **genera la clínica** (receta, cita, estudio) → dueño = Maily Soft. Si lo genera el **paciente** (registro de bienestar, adherencia, ánimo) → dueño = Maily te cuida. Lo clínico es la fuente de verdad; lo de engagement vive del lado del paciente.

CAMBIO 8 · BAJO · BLOQUEANTE

C8. Correcciones de seguridad (antes de cualquier cosa)

Independientes de la unificación, estos deben corregirse en la **primera semana**. Son de bajo esfuerzo pero alto impacto:

#	Problema	Corrección
 A1	Webhook de MailySoft omite la firma si no hay secreto (usa una variable de entorno inexistente) → se pueden falsificar recetas	Fallar cerrado: rechazar si no hay secreto; usar la variable <code>ENVIRONMENT</code> correcta; exigir el secreto en producción.
 A2	El JWT de Clerk no valida <code>iss</code> ni <code>azp</code> → acepta tokens de otras instancias	Validar issuer y authorized-party; cachear JWKS con TTL.
 M1	DSN de Sentry real hardcodeado en <code>base.py</code>	Mover a variable de entorno y rotar el DSN.
 M2	Django 5.1.4 con CVEs conocidos	Subir a la última 5.1.x + <code>pip-audit</code> en CI.
 Bug	<code>notify()</code> no acepta <code>extra_data</code> → <code>TypeError</code> en runtime (notificaciones que nunca se mandan)	Corregir la firma + test de regresión.

CAMBIO 9 · BAJO

C9. Calidad y herramientas

- **mypy + django-stubs en CI** — cazaría el tipo de bug del `extra_data` . Type hints en la capa HTTP (hoy casi sin tipar).
- **Modelo base abstracto** — ya cubierto en C1; elimina la duplicación de `id/created_at/updated_at` en cada modelo.
- **Arreglar N+1** — en `vitals` (una query por tipo de signo vital) y agregar `select_related` / `prefetch_related` en listados con FKs.
- **factory_boy + conftest.py** — ya está instalado pero no se usa; centralizar fixtures de prueba.

- **pyproject.toml** — hoy no existe; agregar config de black, ruff, mypy, pytest en un solo lugar.

SECCIÓN 4

Plan por fases

```
FASE 0 – Seguridad y tooling (semana 1) [BLOQUEANTE]
• C8 (todos los fixes de seguridad + bug extra_data)
• C9 parcial: pyproject.toml + mypy + pip-audit en CI

FASE 1 – Cimientos unificados (semanas 2-4) [EL GRUESO]
• C1 modelo base + multi-tenant (módulo por módulo)
• C4 auth: corregir validación + decidir JWT vs abstracción
• C2 inicio: separar PatientAccount global de ClinicPatientLink

FASE 2 – Reubicación de dominios (semanas 4-7)
• C7 resolver dueño de cada app (recetas/citas/labs → Maily Soft)
• C2 completar: repuntar datos clínicos a patient_link
• C5 cobro dual (separar B2B de B2C premium)

FASE 3 – Especialidades y limpieza (semanas 7-10)
• C3 nutrición como plugin de referencia
• C6 capa de servicios (gradual, donde se toque)
• C9 resto: N+1, factories, modelo base en todos los modelos
```

Estrategia

Igual que la migración del legacy: **módulo por módulo, nunca "big bang"**. Cada fase deja el sistema funcionando. La seguridad va primero porque no depende de nada.

SECCIÓN 5

Decisiones que debes tomar

Decisión	Opciones	Recomendación
¿Unificar de verdad o dos backends?	(a) Un backend unificado · (b) Dos backends que se hablan por API	Si el equipo es pequeño y quieres ir rápido, (b) es más barato. Si quieres una sola base a largo plazo, (a) — este documento.
Auth	(a) JWT propio · (b) Mantener Clerk tras abstracción	JWT propio si Maily Soft también lo usará (consistencia).
Dueño de recetas/citas	Maily Soft (clínica) vs. compartido	Maily Soft es la fuente de verdad; el paciente consume.
Cobro del paciente	Eliminar B2C · Mantener como premium	Mantener como premium (segunda fuente de ingreso).

SECCIÓN 6

Riesgos

- **Migración de datos:** agregar `tenant_id` e introducir el MPI exige migrar datos existentes con cuidado. Doble lectura y validación cruzada.
- **Alcance:** es tentador "arreglar todo de paso". Limitate a las fases; no mezcles refactor de unificación con features nuevas.
- **Romper la app del paciente:** hay usuarios en producción (la primera versión). Cada fase debe mantener la app funcionando — feature flags y despliegues graduales.
- **Seguridad olvidada:** si C8 no se hace primero, el sistema queda explotable mientras se refactoriza. Es lo primero, sin excepción.

Lo único innegociable

Hagas (a) o (b), la **Fase 0 (seguridad)** se ejecuta YA. El webhook que permite falsificar recetas y la validación del JWT no pueden esperar al refactor.